ALGORITHMIQUE

Gbémiga OGOUBY
Architecte Logiciel



Plan du cours

1. Généralités sur les algorithmes
2. Langage algorithmique
3. Déclaration de données
4. Les instructions
5. Les structures de contrôle
6. Les fonctions et les actions
7. Les tableaux

Généralité sur les algorithme

Généralités - Concepts importants

- **Algorithme**

Mot dérivé du nom du mathématicien al_Khwarizmi qui a vécu au 9ème siècle, était membre d'une académie des sciences à Bagdad .

Un algorithme est une méthode de résolution d'un problème sous la forme d'une suite d'opérations élémentaires obéissant à un enchaînement déterminé.

Un algorithme prend des données en entrée, exprime un traitement particulier et fournit des données en sortie.

Généralités - Concepts importants

- **Programme** : série finie et non ambiguë d'instructions pouvant s'exécuter en séquence, ou en parallèle (parallélisme matériel) qui réalise (implémente) un algorithme

Généralités - Pourquoi un algorithme?

- **Pour obtenir** de la «machine» qu'elle effectue un travail à notre place
- **Problème**: expliquer à la «machine» comment elle doit s'y prendre
- **Besoins** :
  - savoir expliciter son raisonnement
  - savoir formaliser son raisonnement
  - concevoir (et écrire) des algorithmes:
  - séquence d'instruction

Généralités - Exemples d'algorithme

- Recette de cuisine
- Notice de montage de meuble en kit
- Mathématiques : problème 3n+1: élémentaire mais redoutable
  - si n est pair, on le divise par 2 ;
  - si n est impair, on le multiplie par 3 et on ajoute 1.
  - Est-il vrai que l'on finira tôt ou tard par tomber sur 1 ?

Généralités - Notion de décidabilité

Une proposition (on dit aussi énoncé) est dite décidable dans une théorie axiomatique, si on peut la démontrer ou démontrer sa négation dans le cadre de cette théorie.

Un problème est décidable s'il existe au moins un algorithme pour le résoudre. C'est-à-dire qu'il existe un nombre fini d'étape pour décider de répondre au problème par l'affirmatif ou par la négation.

Généralités - Notion de décidabilité

En termes plus concrets, cela veut dire qu'on demande au système de fournir une conclusion sans lui avoir fourni suffisamment d'hypothèses.

Ainsi, l'âge du capitaine d'un bateau est indécidable en fonction du tonnage et de la vitesse du navire.

Généralités - Calculabilité

Une fonction *f* est calculable si, pour tout argument *x*, il est possible d'obtenir *f(x)* en un nombre fini d'étapes.

Un algorithme étant assimilable à un calcul, on dira qu'un problème est calculable s'il existe un algorithme pour le résoudre.

NB: La calculabilité n'implique pas toujours un calcul au sens propre du thème.

Généralités - Calculabilité

Classe de problèmes

Les problèmes sont classiquement classés en fonction des algorithmes qui les résolvent :

- **P** : il existe un algorithme qui résout le problème et se termine en temps polynomial, c'est-à-dire qui n'explose pas en fonction de la taille des données d'entrées.

- **NP** : il existe un algorithme qui vérifie la solution d'un problème en temps polynomial mais il faudra un temps exponentiel pour trouver une solution.

- **Indécidables** : Il n'existe pas d'algorithme pour résoudre le problème.

Généralités - Calculabilité

Les problèmes NP

Les problèmes **NP** sont considérés comme impossibles à résoudre dans un temps humain.

Exemple:

Faut-il déplacer ma reine ou mon fou ? Un problème **NP** très connu est celui du calcul du meilleur coup dans le jeu d'échecs.

Pour chaque position aux échecs, il existe un nombre extrêmement grand de positions possibles. Ainsi, il est possible théoriquement de calculer toutes ces positions. Mais l'algorithme mettra des siècles à se terminer.

Les IA calculent toutes les positions possibles sur les *x* prochains tours et choisiront celle qui offre la meilleure position dans *x* tours.

Ce qui rend les intelligences artificielles meilleures que les humains est leur capacité à se projeter plus loin que les humains. Les joueurs humains calculent également les potentielles positions futures mais peuvent calculer beaucoup moins de tours que les ordinateurs modernes.

Généralités – Exigences d'un algorithme

Validité:

Un algorithme est considéré comme valide si le résultat renvoyé est le bon pour toute entrée possible. Il est donc important de bien spécifier le **domaine de définition** des entrées.

Robustesse:

Prise en compte de tous les cas de figure ou conditions anormales d'utilisation.

Exemple: ***la division par 0***.

Généralités – Exigences d'un algorithme

Réutilisabilité:

Un algorithme est réutilisable lorsqu'il peut être utilisé pour résoudre les problèmes équivalents à ceux pour lesquels il a été conçu.

Complexité:

Elle fait référence au nombre d'instruction à exécuter par un algorithme avant d'aboutir à la résolution du problème pour lequel il a été conçu.

Généralités – Exigence d'un algorithme

Efficacité:

Un algorithme est dit efficace lorsqu'il est en mesure d'optimiser les ressources nécessaires à son exécution.

Généralités – Caractéristiques d'un algorithme

- Clarté;
- Instructions élémentaires ;
- Non ambiguïté;
- Précision;
- Effectif;
- Optimal

Exercices

Langage algorithmique

Langage algorithmique

Algorithme NomAlgorithme
{ ceci est un commentaire}
Début
... Actions
Fin

L'algorithme a un nom.

Les blocs d'instruction sont encadrés par
Les mots clés « **Début** » et « **Fin** »

- Il faut avoir une écriture rigoureuse
- Il faut avoir une écriture soignée : respecter l'indentation
- Il est nécessaire de commenter les algorithmes
- Il existe plusieurs solutions algorithmiques à un problème

Déclaration des données

Variable:

Variable <nom_donnee> : **type**

- Instruction permettant de réserver de l'espace mémoire pour stocker des données
- Dépendant du type des données : entiers, réels, caractères, etc.)

Exemples : Variables val, unNombre: **entiers**
nom, prénom : **chaînes de caractères**

Déclaration des données

Constante:

Constante <nom_donnee> : **type** ⟵ **valeur ou expression**

Instruction permettant de réserver de l'espace mémoire pour stocker une constante dont la valeur ne varie pas.

Exemples : Max : **entier** ⟵ 10

DeuxMax: **entier** ⟵ Max * 2

Types de donnée primitifs

- Le type Entier

  ***Ex**: 5; -10; 3*
- Le type Réel

  ***Ex**: 25,5; 17,2*
- Le type Caractère

  ***Ex**: e, d, o, z*
- Le type Chaîne de caractère

  ***Ex**: bonjour*
- Le type Booleen

  ***Ex**: Vrai; Faux*

Les opérateurs

- Opérateurs arithmétiques
  - ***+*** *Addition*
  - ***-*** *Soustraction*
  - ******* *Multiplication*
  - ***/*** *Division*
  - ***DIV*** *Division entière*
  - ***MOD*** *Modulo*

- Opérateurs de comparaison
  - ***<, <=, >, >=*** *Infériorité et supériorité*
  - ***==*** *Egalité*
  - ***!=*** *Différence*

Les opérateurs

- Affectation

  ←

  Ex: a ← 5

- Opérateurs logiques

  OU

  ET

- Opérateur de concaténation : **&**

A	B	A ET B	A OU B
Vrai	Vrai	Vrai	Vrai
Vrai	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai
Faux	Faux	Faux	Faux

Lecture et affichage de données

- Lecture de donnée

  Pour lire une information saisie au clavier par l'utilisateur, on utilise la fonction **Lire**.

  Lire(<nom_variable>)
  Ex: Lire(note)

- **Affichage de données ou d'informations**

  Pour afficher une information en sortie (écran), on utilise la fonction **Ecrire**.

  Ecrire("Chaine", <variable>)
  Ex: Ecrire(''Bonjour Jean'')
  Ecrire(''La somme de '', a, '' et '', b, '' est : '', resultat)

Exercices

Exercices

- Ecrire un algorithme qui calcule le diamètre, le périmètre et la surface d'un cercle à partir de son rayon.
- Ecrire un algorithme qui calcule les coefficients d'une droite à partir de deux points.
- Ecrire un algorithme de résolution d'une équation du second degré admettant exactement deux solutions.

Structures de contrôle

Une structure de contrôle est un bloc d'instructions composé d'actions élémentaires.

1- **Structures de contrôle conditionnelles**

Une structure de contrôle conditionnelle est basée sur la notion de condition ou d'expression booléenne.

Elle utilise une combinaison d'opérateurs de comparaison et d'opérateurs logiques pour déterminer si la condition posée est vraie ou pas.

Structures de contrôle

1-1- **Structure SI ALORS SINON (Structure alternative)**

Elle permet d'effectuer une action si la condition posée est vraie et éventuellement une action alternative dans le cas contraire.

Syntaxe:

```
SI <condition> ALORS
    <instructions_si_vrai>
[SINON
    <instructions_si_faux>]
FINSI
```

Exemples:

```
SI heure > 12 ALORS
    Ecrire(''Bonsoir'')
FINSI
```

```
SI heure > 12 ALORS
    Ecrire(''Bonsoir'')
SINON
    Ecrire(''Bonjour'')
FINSI
```

Structures de contrôle

1-2- **Structure SI imbriquée**

Elle est utilisée dans le cas où on veut vérifier plus de deux conditions énoncés (vrai ou faux) comme dans le cas précédent.

Syntaxe:

```
SI <condition_1> ALORS
    <instructions_si_condition_1>
SINON
    SI <condition_2> ALORS
            < instructions_si_condition_2 >
    SINON
          <instructions_par_défaut>
      FINSI
FINSI
```

Exemple:

```
SI note < 10 ALORS
      Ecrire(‘Passable’’)
SINON
       SI note >= 10 ET note<=14 ALORS
              Ecrire(’’Bien’’)
       SINON
              Ecrire(’’Très bien’’)
       FINSI
FINSI
```

Exercices

- Ecrire un algorithme permettant de déterminer le nombre de solutions qu'admet une équation du second degré.

- Ecrire un algorithme qui permet d'afficher le plus petit nombre parmi trois nombres saisis au clavier.

Structures de contrôle

1-3- **Structure à choix multiple SELON**

Elle est utilisée pour factoriser une action qui se répète pour une variable qui peut prendre plusieurs valeurs. A une liste de valeur, on associe donc une suite d'instructions.

Syntaxe:

```
SELON (<variable>) FAIRE
        Cas <liste_valeurs_1> : <instructions_1>
        Cas <liste_valeurs_2> : <instructions_2>
        ……..
        Cas <liste_valeurs_n> : <instructions_n>
SINON
        <instructions_par_défaut>
FINSELON
```

Structures de contrôle

1-4- **Structure à choix multiple SELONQUE**

Elle est utilisée pour vérifier plusieurs conditions et exécuter une action dans chaque cas. Elle est plus appropriée lorsqu'on ne veut pas imbriquer plusieurs structures SI.

Syntaxe:

```
SELONQUE
        <condition_1> : <instructions_1>
        <condition_2> : <instructions_2>
        ……..
        <condition_n> : <instructions_n>
SINON
        <instructions_par_défaut>
FINSELONQUE
```

Exemple:

```
SELONQUE
        note <= 7 : Ecrire(''Passable'')
        note >= 8 ET note <=12 : Ecrire(''Bien'')
        note >= 13 ET note <=16 : Ecrire(''Très bien'')
SINON
         Ecrire(''Excellent'')
FINSELONQUE
```

Exercices

Structures de contrôle

2- **Structures itératives**

Les structures itératives (boucles) permettent de répéter une instruction ou suite d'instruction un certain nombre de fois.

2-1- **Structure TANTQUE**

La structure TANTQUE permet répéter une suite d'instructions tant que la condition spécifiée est vérifiée. Elle vérifie d'abord la condition puis exécute l'instruction.

Syntaxe:

```
TANTQUE <condition> FAIRE
    <instructions>
FINTANTQUE
```

Exemple:

```
i <- 2
TANTQUE i <= 10 FAIRE
    Ecrire(i*2)
    i <- i + 1
FINTANTQUE
```

Structures de contrôle

2-2- **Structure REPETER**

Elle joue le même rôle que la structure TANTQUE. Sa particularité est qu'elle exécute au moins une fois l'instruction avant de vérifier la véracité de la condition posée.

Syntaxe:

```
REPETER
        <instructions>
JUSQUA <condition>
```

Exemple:

```
i <- 1
REPETER
      Ecrire(i*5)
      i <- i + 1
JUSQUA i <= 12
```

Structures de contrôle

2-3- **Structure POUR**

Cette structure répétitive est utilisée lorsqu'on connaît à l'avance le nombre de répétition à faire. La boucle POUR utilise un compteur qui a une valeur maximale et qui s'auto incrémente jusqu'à la valeur maximale indiquée. On peut également spécifier une valeur de pas différente de 1 qui est la valeur par défaut.

Syntaxe:

```
POUR <compteur> DE <min> A <max>
[A PAS DE <valeur_pas> ] FAIRE
      <instructions>
FINPOUR
```

Exemple:

```
POUR i DE 1 A 50 A PAS DE 5 FAIRE
    Ecrire(i*5)
FINPOUR
```

Exercices

Exercices

- Ecrire un algorithme permettant de déterminer le résultat de n puissance x (n^x).
- Ecrire un algorithme permettant de calculer la factorielle d'un nombre n.
- Ecrire un algorithme qui détermine si un nombre est premier.
- Ecrire un algorithme qui permet de déterminer la n-ième valeur d'une suite de Fibonacci.

Exercices

- Ecrire un algorithme qui détermine et affiche la Nème valeur de la suite (UN) sachant que

  U0 = 0 U1 = 1 ; U2 = 2 ; UN = UN-1 + UN-3 pour N > 2.

- Ecrire un algorithme qui permet à l'utilisateur de saisir une suite caractère se terminant par '*', et qui affiche à la fin le nombre d'apparition de la lettre 'A'.

Fonctions et Actions

1- **Fonctions**

Une fonction est un bloc d'instructions qui permet de factoriser une suite d'instructions visant à effectuer un traitement et à fournir un résultat.

Une fonction reçoit éventuellement des données en paramètre, effectue un calcul et retourne un résultat. Une fonction n'interagit pas avec le programme.

A l'intérieur d'une fonction, on peut également déclarer des variables qui lui sont propres et qui ne sont accessible qu'à l'intérieur de cette dernière.

Fonctions et Actions

1- **Fonctions**

Syntaxe:

Fonction <nom_fonction>([<liste_parametres>]) : < type résultat>
[<Déclaration variables locales>]
Début
    <Instructions>
    retourner résultat
Fin

Exemple:

Fonction PerimetreRectangle(long, larg: réels) : réels
Variables perimetre: réel
Début
    perimetre <- 2*(long+larg)
    retourner perimetre
Fin

Une fonction peut être appelée dans une autre fonction ou dans une action

Fonctions et Actions

2- **Actions**

Une action est un bloc d'instructions qui réalise des traitements et qui peut interagir avec le programme. Une action ne retourne pas de résultat.

Syntaxe:

```
Action <nom_action>([<liste_parametres>] )
[<Déclaration variables locales>]
Début
    <Instructions>
Fin
```

Récursivité

La récursivité peut se définir comme la résolution d'un problème à partir de versions plus simple de lui-même.

Une fonction récursive fera donc appel à elle-même un certain nombre de fois avant d'atteindre le résultat final.

Par exemple, la fonction factorielle se définit récursivement par:

1. La factorielle pour un cas élémentaire (règle de base) : **0! = 1**
2. Définition de la factorielle pour un cas non élémentaire, en fonction de la factorielle d'un cas plus simple (règle récursive):

n! = n * (n-1)! pour n >= 1

Récursivité

```
Fonction facto_rec(n : entier) : entier
DEBUT
    SI n = 0 ALORS
        retourner 1
    SINON
        Retourner n*facto_rec(n-1)
    FINSI
FIN
```

Exercices

Tableaux

Encore appelé variable indicée, un tableau est un type de variable permettant de regrouper et de manipuler plusieurs valeurs d'un même type de base.

Exemple: un tableau de caractères.

1- **Déclaration**

Pour déclarer un tableau, on précise sa taille et son type d'élément. La taille correspond au nombre d'élément que le tableau contiendra.

Syntaxe:

<nom_tableau>[taille]: Tableau de <type_valeur>

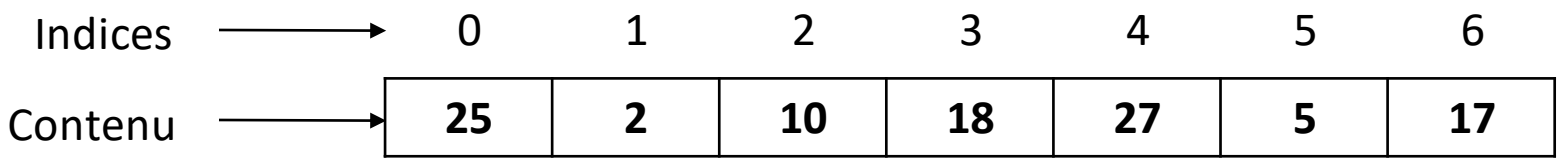
Exemple:

Tab[10]: Tableau d'entiers

Tableaux

2- **Manipulation d'un tableau**

Les éléments d'un tableau son accessibles grâce aux indices. Dans un tableau, les indices partent de 0.



Syntaxe d'affectation:

<nom_tableau>[indice]⟵ <valeur>

Exemple:

Tab[2] ⟵ 15

Syntaxe de lecture:

<nom_tableau>[indice]

Exemple:

Ecrire(Tab[2]) //Affichera 15 en considérant l'exemple précédent